

Mutation-Testing Improvement Loop

IBM Bob's `mutation-climb` skill drives an automated PIT loop that turns surviving mutants into precise test targets, lifting the mutation score from 82% to 95.8%.

82%

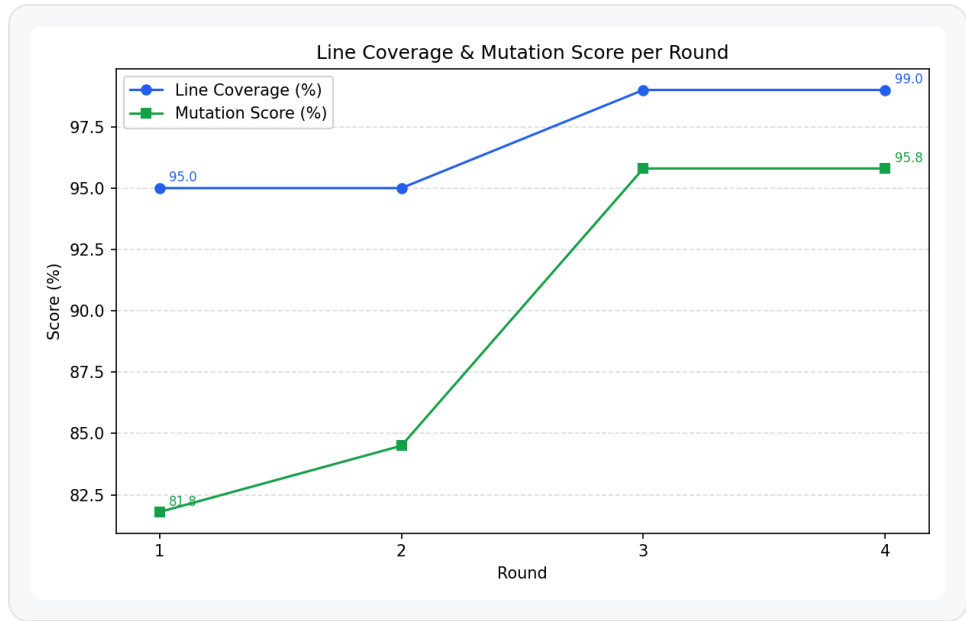
Mutation score before

95.8%

Mutation score after

48 → 11

Surviving mutants



The Problem: Line Coverage Lies

A Java utility library ships with a 95% line-coverage score — green across the board. But line coverage only proves a line was *executed*, not that any assertion actually validates the outcome.

Running PIT mutation testing reveals the real picture: each mutant is a deliberately broken variant of the code. If the test suite doesn't notice, the mutation *survives*. At baseline: **82% of mutants were killed. 18% slipped through.**

The gap: 95% line coverage · 82% mutation score.

Roughly **1 in 5 logic mutations** survives undetected — the lines execute, but nothing asserts the outcome tightly enough.

FILE	LINE	WHY IT SURVIVES
MoneyUtils.java	26	Test covers the line but never hits the exact boundary value
Validators.java	128	Overload is never called — marked NO_COVERAGE
SimpleCache.java	142	Happy-path test never exercises the expiry branch



The Approach: Automated Kill Loop

Each round is driven by a single shell command. IBM Bob's **mutation-climb skill** runs the loop autonomously — reading survivors, writing targeted tests, and re-running PIT until the score plateaus or the target is reached.

- 1

Run PIT via `sh run_round.sh`

Compiles the project, executes mutation analysis, writes raw XML to `pit-output.txt` and copies it to `toolkit/mutations.xml`.
- 2

Parse survivors

`report.py` converts `mutations.xml` into `survivors.md` and appends a row to `rounds.csv`.
- 3

Bob reads survivors and writes targeted tests

Each surviving mutant is a concrete, precise test target — boundary assertion, missing overload call, or time-advance test for expiry logic.
- 4

Repeat until score plateaus or target is reached

Every round `survivors.md` shrinks and `chart.png` updates. Goal: mutation score $\geq 90\%$.

Why this works:

PIT survivors are not vague "untested paths" — they are exact file, line, and mutation-type tuples. Bob can read the tuple, read the source, and write the one assertion that kills it. No guesswork, no redundant tests.

FILE	ROUND-BY-ROUND OUTPUT
<code>survivors.md</code>	Ranked list of still-alive mutants with file, line, and type
<code>rounds.csv</code>	Append-only log of line coverage and mutation score per round
<code>chart.png</code>	Visual progress curve across all rounds; regenerated by <code>chart.py</code>

How IBM Bob Is Used

Three Bob capabilities work together to make the loop autonomous: a domain skill, a custom mode, and parallel subagents.

SKILL

mutation-climb

A Bob skill stored at

```
.bob/skills/mutation-climb/SKILL.md
```

. It

encodes the domain knowledge: how to read

```
survivors.md
```

, how to rank mutants by kill

probability, and which test patterns map to which mutation types (conditional-boundary, negated-conditional, return-value substitution).

Activated at the start of a session — Bob loads the full protocol and proceeds without further prompting.

```
/mutation-climb
```

CUSTOM MODE

Test Engineer mode

A lean Bob mode that restricts the tool palette to exactly what mutation testing needs: file reads, grep, test edits, and the Maven command runner. It prevents Bob from touching production code or files outside `src/test/java/`.

The mode sets a role definition that primes Bob to think like a test author focused on assertion quality, not coverage percentage.

```
mode: test-engineer
```

SUBAGENTS

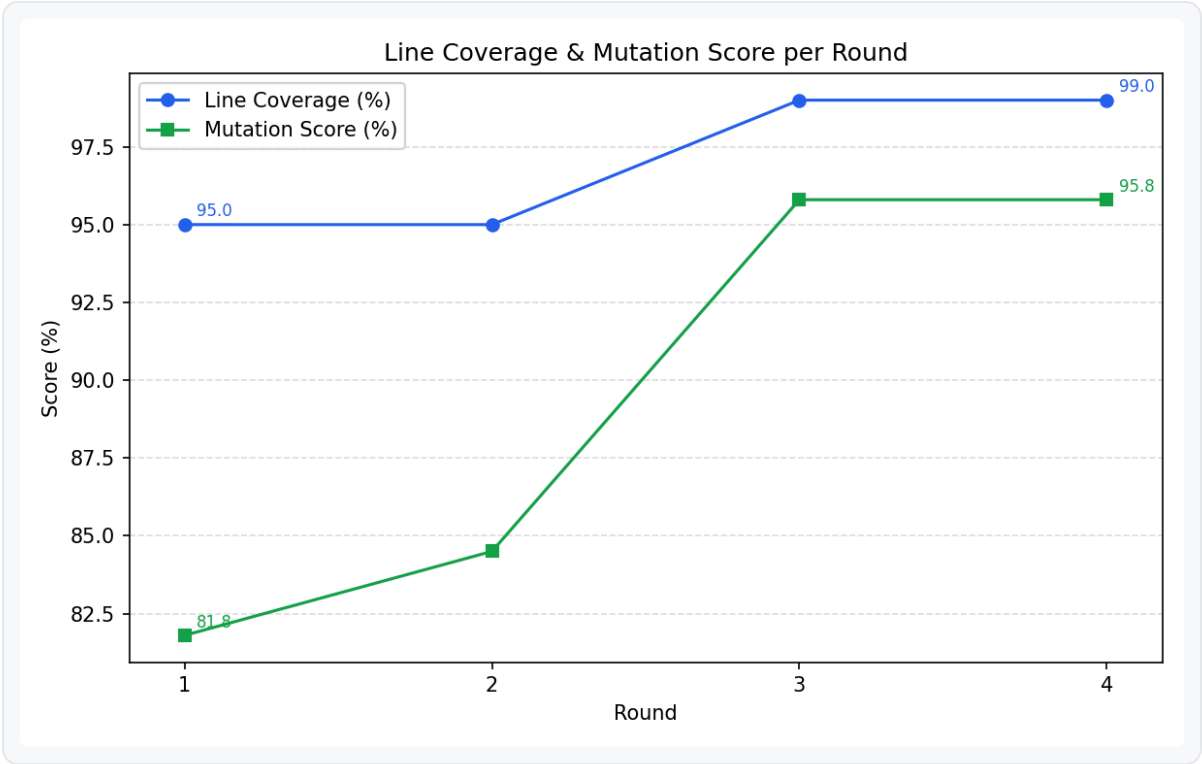
Parallel survivor analysis

When a round produces a large survivor list, Bob spawns independent subagents — one per class under test (`MoneyUtils`, `Validators`, `SimpleCache`, etc.). Each subagent reads its class, reads the surviving mutants for that class, and returns a list of targeted test patches.

The parent agent merges the results and writes a single coherent test file — parallel analysis, serial commit.

```
spawn_subagent
```

Results: 4 Rounds, +14.0 pp Gain



All numbers sourced from `rounds.csv` (round 4) and `survivors.md`. Mutant total held constant at 264 throughout — the library code never changed, only the tests. Every figure reproduces from a clean clone.

	BASELINE	ROUND 4
Line coverage	95%	99%
Mutation score	82%	95.8%
Mutants generated	264	264
Survivors	48	11

+14.0 pp mutation score improvement over 4 measured rounds. More than **77% of previously surviving mutants** are now killed (48 → 11). Line coverage climbed from 95% to 99%.

What Is Next

The loop proved the concept on a small utility library. The following steps scale it to real-world projects and make the skill generally reusable.

- **Close the last killable gaps**

Seven of the 11 remaining survivors are equivalent mutants that no test can kill. The other four, including two `NO_COVERAGE` methods in `Validators`, are the next targets.

- **Extend to other languages**

The skill protocol is language-agnostic at the Bob layer. Swap PIT for Stryker (JS/TS) or mutmut (Python) and the loop runs unchanged — only the `run_round.sh` adapter differs.

- **Survivor-based PR review mode**

A Bob custom mode that reads the diff of an incoming PR and flags any new code path not covered by a mutation-killing assertion — catching test-coverage debt at review time, not post-merge.

- **Publish mutation-climb as a shared Bob skill**

Package the skill with a generic `run_round.sh` template so any Java Maven project can adopt the loop with a single copy-paste.

- **CI integration**

Add a GitHub Actions workflow that runs PIT on every pull request, comments the mutation delta, and blocks merge if score regresses below the previous round's baseline.

- **Benchmark Bob models on test quality**

Use the reproducible `rounds.csv` metric as a cross-model benchmark — run the same survivor list through multiple models and compare how many mutants each kills per round.